

Database Design and Implementation Project

A database architecture for movie review aggregation

POPCORN_INDEX

Table of Contents

Section 1: Introduction	
.....	4
Section 2: Database Plan: A Schematic View	
.....	5
Section 3: Database Design: A Normalized View	
.....	10
Section 4: Database Views	
.....	15
Section 5: Procedural Elements	
.....	18
Section 6: Sample Queries: Database in Action	
.....	21
Section 7: Conclusion	
.....	22
Acknowledgement	
.....	24
References	
.....	25

List of Figures

Figure 1: *ER Diagram of the database*

..... 10

Figure 2 (a) & (b): *Sample records from Scores and Reviews table*

..... 12

Figure 3: *Output of weighted_score view demonstrating critic-influence-based aggregation of reviews*

..... 17

Figure 4: *Query output showing average scores for each film in descending order.*

..... 21

1. Introduction

The POPCORN_INDEX database is developed as the back-end data structure of a movie review aggregation platform. The database system only manages the storage of data, the enforcement of data integrity, and the ability to perform queries on that data in a way that reflects real-world applications such as Rotten Tomatoes. User identification, user interfaces, and recommendation services other than those which can be implemented using SQL and views are not managed by the database system itself.

The main objective of the database is to depict the relationship between films, critics, and the reviews within the review environment. The same film may be a part of a franchise, cross-genre, and have reviews by different critics with different degrees of influence. Each of the reviews has not only a text opinion but also a sentiment classification and numerical score, which enables both qualitative and quantitative analysis. This results in designing the database in such a way that the data is stored in more than one table. This approach aligns with the principles of relational database design, which emphasise minimising redundancy and improving data integrity (Codd, 1970).

As of now, there are around 100 movies in the database, with about 15 critics and approximately 200 reviews. Typically, each movie has more than one review associated with it, thus making aggregation possible. In addition to structured data such as IDs, ratings, and date stamps, the database also has some semi-structured data like the text from the reviews. Though the amount of data is only moderate at the moment, the design is scaled, so the addition of more films, critics, and reviews is a natural increase. This reflects modern database system design, where flexibility and long-term maintainability are essential considerations (Hellerstein et al., 2007).

The database serves the purpose of being the authoritative source of truth in all the data pertaining to the review in the larger system. It enforces consistency (such as avoiding the review of the same critic more than once), validation of input (such as making sure that scores are within valid ranges), and facilitating efficient retrieval of aggregated information. As a result, the database guarantees the consistency of calculations for any metrics calculated based on the data, eliminating the necessity to duplicate logic in the application layer. The attributes like the hit score, influence of critics and trending movies are not programmed into an application layer but rather are directly facilitated by SQL views and functions. This design reflects modern data

management approaches, where computation is increasingly performed closer to the data to improve efficiency and consistency (Stonebraker et al., 2018).

Another important design choice that was made in this project was to include the procedural aspects of the design by including triggers and stored functions as a way of enforcing business rules at data base level. As an example, triggers are employed to check review dates against film release dates, and to guard against invalid score entries. This ensures the integrity of data irrespective of the way the database is accessed. This approach is consistent with established principles of database system implementation, which emphasise integrity enforcement as a core responsibility of the database engine (Hellerstein et al., 2007).

Overall, the POPCORN_INDEX database is made to be balance normalisation, data integrity, and usability. The structure will reduce redundancy but will still be able to support complex queries that are needed by a review aggregation platform. In the sections that follow, the conceptual design, relational structure and implementation decisions will be detailed, and the reasons behind each significant decision will be provided.

2. Database Plan: A Schematic View

The overall design of POPCORN_INDEX database focuses on the conceptualisation of the interactions among the films, critics, and reviews in a manner that facilitates storage as well as aggregation of the data (Chen, 1976). Instead of looking at reviews as isolated entries, the design captures the greater ecosystem within which films are produced, reviewed, and categorised.

2.1 Core Entities

The main entities found in the system are:

- Feature(feature_id, title, release_year, type)

Represents a film. Important attributes are feature-id, title, release year, and type. This is the center or node of the database since most of the other entities are related to it either directly or indirectly.

- Critic(critic_id, name, publication, is_top_critic)

Refers to those who post reviews. Attributes are critic_id, name, publication and is top critic. This enables the system to make a distinction between professional and general users.

- Review(review_id, feature_id, critic_id, review_text, review_date, sentiment)

Is a review by a movie critic. Attributes are review_id, feature_id, critic id, review text, review date and sentiment. This entity resolves the many-to-many relationship between features and critics.

- Score(review_id, score_value, max_score)

Stores the number rating of a review. It contains score value and max score. This was decoupled with the Review entity in order to be able to flexibly deal with various scoring systems.

2.2 Supporting Entities

In order to add richness and complexity to the dataset, a number of supporting entities is added:

- Franchise and Franchise_Features

These entities represent the relationship between movies and franchises. As a film can be part of a franchise and that a franchise contains many films, a junction table (franchise_features) is employed to reflect this many-to-many relationship, and an ordering attribute (franchise number).

- Feature_Genres

Indicates the genres of each movie. This models a many-to-many relationship because a film can belong of many genres, and a genre can be used by many films.

- Actor and Feature_Actors

These components capture casting data. The number of actors in a film and actors in a number of films is that there is a junction table, with a role attribute added to define the role played.

- Feature_Release

The information that is released by stores includes the release date and country. This is not bound to the Feature entity to enable it to be flexible in its modelling of various release contexts.

- Critic_Influence

Normalizes weighted scores by associating a weight with each critic. This is in line with the reality of the world where some critics are more influential than others.

- Review_Votes

Gets user feedback on reviews (e.g., useful or not). This adds another level of interaction, other than critic reviews.

The primary key of each junction is constructed using a combination of the foreign keys of the participating tables. This makes the relationship unique and avoids duplicating an association.

2.3 Relationships Between Entities

The main relationships of the system are:

- A Review is allowed to belong to exactly one Feature but a Feature can have many Reviews (one-to-many).
- A Review may have many Votes, representing user interactions (one-to-many).
- A Critic may write numerous Reviews, but only each Review is written by exactly one Critic (one-to-many).
- One Score is associated with one Review, making it the one-to-one relationship.
- A Feature can also be a part of more than one Genre, and a Genre can apply to more than one feature (many-to-many).
- One Feature may be a part of a Franchise and franchises have many features (many-to-many through a junction table).
- A Feature may contain more than one Actor and Actors may occur in more than one feature (many-to-many).
- One Critic has an associated Critic influence value (one to one).

Also, there are constraints to participation which are incorporated into the design. For instance, all Reviews must have exactly one Feature and Critic each while a Feature or Critic may not have any Reviews at all (one-to-many relationship).

2.4 Design Motivation

One of the objectives of the design was to prevent redundancy and still be flexible. The design does not store all the information in a single table, but separates the concerns into separate entities:

- Scores are decoupled with the reviews so that they can be scored using different systems without affecting the structure of the reviews.
- Junction tables are used to describe many-to-many relationships by using the genre, actors, and franchises.
- The critic influence is kept separately to aid weighted aggregation without redundancy.
- Extensibility was another factor to consider. The schema is built in such a way that it is possible to add new features (e.g., new metadata, new types of interaction or new scoring systems) without rearranging the existing tables.

These decisions align with established database design practices that emphasise modularity, scalability, and maintainability (Hellerstein et al., 2007).

An alternate schema might have involved including the score value and critic weight as attributes of the Review or Critic class. This solution was abandoned due to its tendency to create redundancy and rigidity, as well as making future additions, like adding new scoring or weighting schemes, more difficult.

2.5 ER Diagram

To visually represent the conceptual schema, an Entity-Relationship (ER) diagram is constructed using Chen notation to depict the entities, attributes, and relationships, including cardinalities and participation constraints (Chen, 1976).

The diagram highlights:

- As the key element, feature.
- Review as the connecting body between film and critics,
- Many-to-many relationships that are solved using junction tables,
- Supporting entities that enhance the analysis of the database.

The POPCORN_INDEX database ER diagram has the Feature entity as its central focus and this entity depicts individual movies. This entity serves as the central point of the system since majority of other entities directly or indirectly are associated with it. Attributes used to describe each feature include title, release year, type, and are the major focus of reviews, classifications, and associations in the database.

One of the main relationships in the design is the one between Feature and Critic that is modelled as a many-to-many relationship that is resolved over the Review entity. This is how the experience in the real world is, in that a single movie can be reviewed by various critics and a critic may review various movies. The Review entity contains qualitative and structured data, such as the review text, sentiment and date of review. The design prevents redundancy by making Review a stand-alone element and not a part of Feature or Critic, and allows scalable data expansion.

A one-to-one relationship between Review and Score is also included in the diagram, with numerical ratings being separated out of the textual reviews. This design can be flexible in managing various scoring systems, and makes scoring logic able to change without changing review content. Likewise, the Critic_Influence entity adds to the Critic entity with some additional weights to allow more advanced analysis like a weighted average score without replicating critic data.

The design has several many to many relationships that are settled with the help of associative entities. As an illustration, FeatureGenres is a model of the relationship between films and genres, where a film can be a member of more than one genre. Franchise_Features maps the association between films and franchises as well as maintaining the sequence of movies in a franchise. Similarly, Feature_Actors is a film cast, and has another role attribute to describe character information. These associative entities are used to make sure that the database is well normalised and is capable of giving a real-world relationship.

Other supporting organizations add more to the dataset. The Feature release entity contains release specific data, like date of release and country, which has been moved out of the Feature entity to ensure it is not fixed. The Review_Votes entity brings about user interaction as a user can vote more than once on a review hence the element introduces a one-to-many relationship with Review. This allows the system to get feedback of the audience as well as criticism.

In general, the ER diagram illustrates a design that emphasizes clarity, normalisation and extensibility.

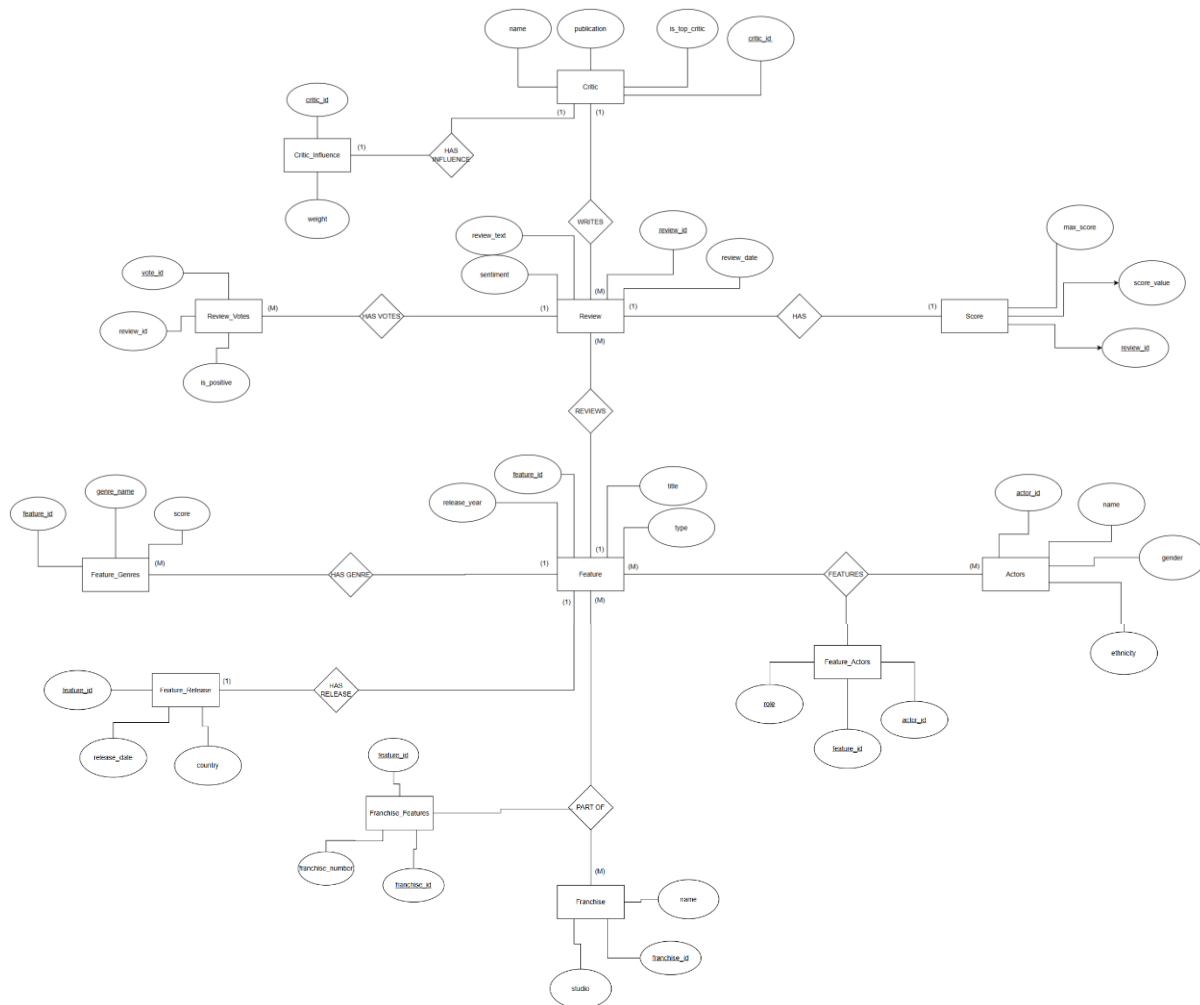


Fig 1 – ER Diagram of the database

Source: Author created using Draw.io

3. Database Design: A Normalised View

The POPCORN_INDEX database is implemented in terms of conceptual design in which the database is represented as a set of relational tables, each depicting a unique entity or relationship in the ER model. The schema has been designed to guarantee data integrity, lessen redundancy and support effective querying.

The relational schema reflects established principles of database normalisation and dependency management, ensuring that the database remains consistent and scalable under increasing data volume (Fagin, 1977).

The relational schema of this database is as follows, the primary keys and foreign keys are denoted as PK and FK, respectively:

Feature(feature_id PK, title, release_year, type)

Critic(critic_id PK, name, publication, is_top_critic)

Review(review_id PK, feature_id FK, critic_id FK, review_text, review_date, sentiment)

Score(review_id PK/FK, score_value, max_score)

Genre(genre_id PK, genre_name)

Feature_Genres(feature_id FK, genre_id FK, PK(feature_id, genre_id))

Franchise(franchise_id PK, franchise_name)

Franchise_Features(feature_id FK, franchise_id FK, franchise_order, PK(feature_id, franchise_id))

Actor(actor_id PK, actor_name)

Feature_Actors(feature_id FK, actor_id FK, role, PK(feature_id, actor_id))

Feature_Release(feature_id FK, country, release_date, PK(feature_id, country))

Critic_Influence(critic_id PK/FK, influence_weight)

Review_Votes(vote_id PK, review_id FK, vote_type)

3.1 Major Tables and their functions

Information about films is stored in the Feature table which is the central table in the database. It has features like, feature_id, title, year of release and type. This table is referred to by most other tables using foreign keys.

The Critics table stores the details of reviewers, their identity, publication and whether the reviewer is considered a top critic. This enables the system to differentiate between various kinds of reviewers.

Critics and features are linked in the Reviews table. Each record is a singular review done by a critic of a particular film. It incorporates attributes like review-text, review-date and sentiment and the foreign key connecting with Feature and Critics.

The one-to-one mapping between the Review and Score tables is achieved through the use of review_id as the primary and foreign key for the Score table. This helps in ensuring that there is a maximum of one score for each of the reviews listed. The design provides some flexibility in dealing with various scoring schemes by decoupling scores and reviews and makes sure that numeric evaluation is addressed separately.

1254 • `SELECT * FROM scores LIMIT 5;`

1255

review_id	score_value	max_score
1	9.5	10
2	6.0	10
3	9.0	10
4	6.5	10
5	9.2	10
NULL	NULL	NULL

1255 • `SELECT * FROM reviews LIMIT 5;`

1256

review_id	feature_id	critic_id	review_text	review_date	sentiment
1	1	1	A landmark cinematic achievement with powerfu...	2020-01-01	positive
2	1	11	Slow pacing but still compelling overall	2020-01-02	negative
3	2	2	A masterful sequel that deepens the original story	2020-01-03	positive
4	2	12	Overly long but well acted	2020-01-04	negative
5	3	3	One of the greatest superhero films ever made	2020-01-05	positive
NULL	NULL	NULL	NULL	NULL	NULL

Fig 2 (a) and (b) Sample records from Scores and Reviews table, respectively.

3.2 Supporting Tables

More complex relationships are modelled by a number of supporting tables:

- Franchises and Franchise_Features are the connection between films and franchises, and the sequence of films of a franchise.
- Feature_Genres enables the films to be linked to several genres.
- Actors and Feature_Actors model cast data, such as the roles that each actor performs.
- Feature_Release contains release-related data, including release date and country.

- Critic_Influence gives weights to the critics to do weighted scoring.
- Review_Votes captures the user feedback on reviews.

These tables ensure that complex relationships are represented without adding redundancy in the main entities.

3.3 Integrity Constraints

The database has a number of constraints that guarantee the validity of the data:

- Each table has primary keys that are used to identify the records in the table.
- Foreign keys ensure that there is referential integrity between tables.
- NOT NULL constraints- These are used to ensure that important attributes are always given.
- CHECK constraints (e.g., sentiment values) limit data to valid categories.
- Triggers also impose other rules like valid score ranges, avoidance of duplicate reviews and valid review dates.

Examples of such constraints from the database are:

primary key (feature_id, franchise_id),

foreign key (franchise_id) references franchises(franchise_id)

check (sentiment in ('positive', 'negative'))

3.4 Indexing Strategy

Besides logical correctness and normalisation, POPCORN_INDEX database is introduced with an indexing strategy to enhance the performance of the queries. With increasing number of reviews and other data, it becomes very important to have an efficient way of retrieving data, especially when you need to aggregate intensive data in a query like the hitmeter, weighted score and trending movies views.

The Reviews and Scores tables have indexes created on their commonly used attributes. In particular, review(feature_id) and review(critic_id) indexes optimise the join between Reviews, Features, and Critics tables which are the focus of most analysis queries. The reviews (review

date) index facilitates filtering by time, especially in the trending films view where recent activity is computed by date range.

A score(review_id) index enhances the efficiency of a join between the Scores and Reviews tables, and is necessary to compute average and weighted scores. Further, a composite index on reviews (feature_id, critic_id) is added to optimise queries that filter or validate at once on both attributes, e.g. find out if a review is a duplicate, or get critic-specific information about a movie.

These indexing choices are in line with the query patterns of Section 6 and the views of Section 4. The indexing strategy will make the database performant and scalable even as the size of the dataset grows since it lowers the cost of joins and filtering operations. Although indexes add a minor overhead to the insert and update operations, this is a worthwhile trade-off since a review aggregation system has a read-intensive nature.

3.5 Normalisation Overview

The database will meet the needs of the First, Second and Third Normal Forms.

- The First Normal Form (1NF) is obtained because of the absence of repeating groups, and all the tables have atomic attributes. Associative tables are divided into multi-valued data, like genres and actors.
- The Second Normal Form (2NF) is met because all non-key attributes are based on the entire primary key. This is especially clear from associations tables such as Feature_Genres and Feature_Actors that utilize composite primary keys. The other columns, like role in Feature_Actors, depend completely on the composite key and not just parts of it, which satisfies the Second Normal Form criteria.
- Third normal form (3NF) is obtained by eliminating transitive dependencies. For instance, attributes like score are placed in the Score table and not in the Review table to ensure that non-key attributes depend solely on the primary key and not on other non-key attributes.
- The database is also in Boyce-Codd Normal Form (BCNF), since all the functional dependencies are on the candidate keys (Bernstein, 1976). For instance, in the table Review, the functional dependency will be:
review_id → feature_id, critic_id, review_text, review_date, sentiment

As review_id is a candidate key, all attributes in the non-key column are fully functionally dependent on it, satisfying BCNF. This holds true for other tables as well, where all the determinant columns are candidate keys.

3.6 The normalisation of POPCORN_INDEX

To illustrate how POPCORN_INDEX database can be normalised, one can consider the analysis of the data requirements in the real world and the breakdown of the data requirement into several related tables to remove the problem of redundancy and dependency.

A good illustration here is the isolating of Feature, Genre and Feature Genres. Rather than having various genres stored in the Feature table (which would break 1NF because there would be duplicate groups), the genres are stored in a different relational structure. This many-to-many relationship is solved in the Feature_Genres table whereby each row is composed of atomic values and a feature may have several genres. This ensures atomic values and eliminates repeating groups, satisfying the requirements of First Normal Form (Kent, 1983).

Likewise, Feature and Critic table is normalised by the Reviews table. Instead of repeating information about a critic or features in several records, the Review entity only contains foreign keys to the other tables. This removes redundancy and makes sure that all non-key attributes in Reviews are totally dependent on the primary key (review_id) and they meet the requirements of the Second Normal Form.

Further evidence of normalisation is seen in the fact that the Scores table has been separated into Reviews table. When the score values directly were stored in Reviews, this would add extraneous dependencies and decrease flexibility. The design eliminates transitive dependencies and accommodates the Third Normal Form by isolating score-related attributes into an independent table with a one-to-one relationship.

The other example is found in the Critic_Influence table. A Critics weight is not stored in the Critics table, but instead it is stored in its own relation. This makes sure that modifications to critic influence do not propagate to unrelated critic attributes and not to cause any hidden dependencies, which is consistent with Third Normal Form, and with the BCNF principles.

Lastly, the associative tables like Feature_Actors and Franchise_Features are the ones that exhibit the correct way of dealing with composite keys. The attributes in these tables rely on the

complete composite key, and there are no partial or transitive dependencies. This proves that not only does the database structure comply with 2NF and 3NF, but also complies with the more rigorous BCNF requirements in practice.

4. Database Views

To allow easy and meaningful access to aggregated data based on the underlying relational tables, the POPCORN_INDEX database has a number of SQL views. These views can be used as an abstraction layer, where the application can access complex analytical results without the need to re-run detailed queries. Each perspective is intended to assist a particular application in a movie review aggregation system. SQL views are commonly used to abstract complex queries and present simplified, user-oriented representations of underlying data structures (Hellerstein et al., 2007).

Each view is tailored for a particular target user or component that utilizes it, including end users viewing movies, analysts analysing trends, and system components calculating ratings.

4.1 Hitmeter View

The hitmeter view determines the percentage of positive reviews of each feature. This can be compared to a freshness score in the real-world platforms. Measures like hitmeter scores look homogenous due to the controlled dataset where there are the same number of positive and negative reviews on a film. This is a weakness of the sample data, but not the database design.

This is mainly aimed at the end users where it gives a quick and easy judgment of the reception of a film. It will use sentiment data as a raw review and convert it into a meaningful metric by aggregating the data. It is done using a view to prevent storing derived data and to keep the results current. Saving it in the form of a table would create redundancy and would need regular updating with the addition of new reviews.

For example, an application could query this view to display highly rated films:

```
SELECT * FROM Hitmeter WHERE hit_score > 75;
```


4.2 Certified Fresh View

The `certified_fresh` view filters movies with a set quality level (60% positive reviews). This is a replica of the real-life categories like Certified Fresh. There are no films in the current dataset which have a 60% threshold because the negative and positive reviews are evenly distributed, but the view remains valid, and illustrates how such classifications would work with more heterogeneous data.

The view can be applied in front-end filtering and recommendation systems so that the application can highlight highly rated films. It is described as a view since it is based on the score of the hitmeter, which is also derived data. Saving this classification alone would cause incongruency in case of changes in the review data.

An application might use this view as follows:

```
SELECT * FROM Certified_Fresh;
```

4.3 Feature Average Score View

The feature average score view finds the average numerical score of each feature. Analytical queries and ranking systems are favoured by this view, allowing films to be compared based on critic rating and not just sentiment. It is applied as a perspective since average scores are constantly evolving values, contingent on reviews and scores coming in.

For instance, films can be ranked using:

```
SELECT * FROM Feature_Average_Score ORDER BY avg_score DESC;
```

4.4 Weighted Score View

The weighted-score view builds on the calculation of average scores by adding weights on the influence of critics. This view is especially applicable in the case of advanced analytics where more influential critics can have a stronger impact on the ultimate score. It is considered a view since the weighted average is calculated dynamically and relies on numerous tables. It would be a breach of normalisation to store it separately and cause some redundancy.

This enables queries such as:

1253 • `SELECT * FROM weighted_score LIMIT 5;`

1254

Result Grid | Filter Rows: | Export: | Wr

	feature_id	weighted_avg
▶	1	8.1000000
	11	7.2800000
	21	7.5800000
	31	7.5800000
	41	7.2800000

Fig 3 Output of weighted_score view demonstrating critic-influence-based aggregation of reviews

4.5 Trending Films View

The trending_films view is the view that determines the films that have had the most recent reviews within 30 days. This opinion promotes the real-time information and interactivity features, including showing the movies that are now popular or trending. It is put in place as a view since it relies on time sensitive data and should always give the latest state of the database. The output looks homogenous due to the controlled dataset, this is a weakness of the sample data, but not the database design.

An example use case would be:

```
SELECT * FROM Trending_Films ORDER BY recent_review_count DESC;
```

4.6 Design Choices Overview.

Some major advantages associated with the use of views in the POPCORN_INDEX database include:

- They contain the logic for complex aggregations which can be used repeatedly throughout the database.
- The views allow separation of computations from the data and therefore retain normalization.
- They keep the calculated data, such as scores and trends, up-to-date and do not need updating by the user.
- The views enable custom access for different users, ranging from end-users to analysts and applications.

Generally, the use of views makes the database easier to work with because the data becomes more understandable.

5. Procedural Elements

The POPCORN_INDEX database has a number of procedural components that are implemented as triggers and stored functions. They are used to impose business rules, and ensure data integrity that cannot be implemented solely by standard constraints. Although primary keys, foreign keys and CHECK constraints are used to do basic validation, procedural elements are used to guarantee more complex, real-world constraints are always upheld at the database level. The procedural features, which include triggers and stored functions, are popularly implemented in relational databases to impose business rules and provide additional data integrity on top of normal constraints (Widom, 1995).

5.1 Score Validation Trigger

This trigger makes sure that the values of the scores are valid. It will avoid the entry of invalid or inconsistent data into the Scores table.

Such validation could not be implemented entirely with the standard CHECK constraint especially when it was used to compare two attributes (score_value and max_score). The trigger is such that all the scores are logically sound when they are inserted.

5.2 Duplicate Review Prevention Trigger

This trigger helps to stop a critic posting more than one review of the same feature. It puts into practical force a rule that every critic must never review a film twice.

Though this restriction might be imposed with a composite unique constraint, the trigger offers a more adaptable and explicit system, with the ability to have customised error handling and can be extended in the future.

5.3 Review Date Validation Trigger

This trigger will prevent reviews that are submitted prior to the official release of a movie. It provides time consistency between related tables.

The validation cannot be provided only with the help of standard constraints because this validation presupposes the necessity to query a second table (Feature_Release) in case of insertion. The trigger is used to keep all the data of the reviews consistent with release information.

5.4 Stored Function: Score at Hitmeter

This operation computes the quantity of positive reviews of a feature. It summarizes logic like the hitmeter view, except that it can be applied programmatically as part of queries.

For example:

```
select title, get_hitmeter_score(feature_id)
from features;
```

This operation can be used to design queries in a modular way, such that the same logic can be reused without having to rewrite complicated aggregation queries.

5.5 Procedural Elements Justification

This database enforces rules which cannot be represented completely in declarative SQL constraints alone by the use of procedural elements. In particular:

Triggers enable validation by many attributes or tables,

They make sure that rules are automatically enforced at the time of insertion,

They ensure that invalid data does not ever make it into the database.

In the absence of these procedural aspects, the task of implementing such rules would be left to the application layer, which would risk greater inconsistency.

Stored function is also very useful as it provides a better usability by bundling the commonly used logic into a reusable unit.

5.6 Summary

Inclusion of triggers and functions makes the POPCORN_INDEX database more robust as it will make sure that the information is valid and consistent, business regulations are centrally implemented, and complex logic is maintainable and reusable.

Such procedural components supplement the relational design and serve a major role to support the reliability of the system.

6. Sample Queries: Database in Action

The POPCORN_INDEX database is to facilitate a movie review aggregation system. The queries below illustrate how the database could be utilized to produce useful information to the end-users, critics, and the application itself. Each query is a real-world application and is executed upon the populated data.

6.1 Ranking Movies by Hitmeter Score

Purpose: The query brings back all the movies according to their Hitmeter rating, which is the percentage of positive reviews.

Use Case: That is what the application actually does - it shows rankings just like the sites such as Rotten Tomatoes. It enables users to find films that are critically successful fast.

Insight Provided: Users are able to immediately view the most positively received films.

6.2 Mean Numerical Rating per Film

Purpose: This question is used to compute the mean of the numeric rating given to each movie.

Use Case: The Hitmeter offers a binary sentiment rating, whereas this query offers a more specific rating of the opinion of critics.

Insight Provided: It emphasizes movies that not only get good critiques but are also rated highly in regards to their scores.

1226	•	select
1227		f.title,
1228		avg(s.score_value) as avg_score
1229		from features f, reviews r, scores s
1230		where f.feature_id = r.feature_id
1231		and r.review_id = s.review_id
1232		group by f.feature_id, f.title
1233		order by avg_score desc;

Result Grid		Filter Rows:	Export:	Wrap Ce
	title	avg_score		
▶	The Godfather	7.75000		
	The Godfather Part II	7.75000		
	Forrest Gump	7.50000		
	X-Men: Days of Future Past	7.45000		
	The Shawshank Redemption	7.45000		
	Captain America: The First Avenger	7.45000		
	Frozen II	7.45000		
	The Lord of the Rings: The Two Towers	7.45000		
	Inception	7.40000		

Fig 4 Query output showing average scores for each film in descending order.

6.3 Highest Active Critics

Purpose: Shows critics who have made the most reviews.

Use Case: This may be applied to draw out the prominent or the very active critics in the system.

Insight Provided: The app can rank or place prominence to critics who post regularly.

6.4 Top Critic Per Film

Purpose: Counts the number of reviews of each movie by leading critics.

Use Case: Other websites place greater weight on top critic reviews. This question is what makes this distinction possible.

Insight Provided: Users are able to distinguish between general consensus and expert opinion.

6.5 Summary

The above queries illustrate the extent to which POPCORN_INDEX database facilitates many application functions, such as:

- Discovery and ranking of films.
- Critic activity analysis
- Sentiment and scoring analysis.
- Trending content detection

- Weighted expert evaluation

Every query is executed on the defined schema and filled up data which makes it possible to get the meaningful and realistic results. These questions combined depict the role of the database as the foundation of a movie review aggregation system.

7. Conclusion

The POPCORN_INDEX database shows how an appropriately normalised relational schema may serve the main purpose of a movie review aggregation site. The design focuses on the Feature, Critics, and Reviews tables with the Reviews relation completing the many-to-many relationship between the films and the film critics using well-defined foreign keys. This division of Scores and Reviews makes it possible to do flexible numerical analysis, and the Critic_Influence factor may be aggregated using weights, without redundancy. Analysis capabilities like the hitmeter and weighted score views and the function `get_himeter` score, demonstrate how insightful information may be obtained directly at the database level.

One of the strengths of the implementation is the incorporation of data integrity mechanisms both declaratively and procedurally. Primary and foreign keys enforce referential consistency, and triggers (e.g. not allowing duplicate reviews and checking review dates against the Feature_Release table) enforce domain-specific constraints that cannot be expressed using conventional SQL constraints. This combination guarantees that the database can be strong irrespective of its access or extension.

Nevertheless, the design entails some trade-offs. As an example, the normalisation of objects like Scores and Feature_Release, which is hard, enhances flexibility and consistency, but adds more joins to a common query, which may affect performance at scale. Likewise, the decision to store sentiment as a constant in the Reviews table makes it easier to query, but reduces the flexibility to more complex or multi-dimensional sentiment analysis.

Furthermore, the existing implementation is that the dataset used is controlled and relatively homogenous with every feature having the same number of positive and negative reviews. Consequently, some of the results of the analysis, including the scores of hitmeter and the trending indicators, seem homogenous and do not fully reflect the variability witnessed in real-

life situations. This is not a structural constraint and the database design can always be expanded to handle more varied and realistic data without alteration.

The schema is however well placed to be developed in future. The system could be expanded to include a user's entity that would enable personalised engagement with the system, including saved movies or rating movies personally. To incorporate a recommendation system, a linking table (e.g., User_Preferences) may be added, as well as to use the established relationships between Feature and Feature_Genres and Reviews to make similarity-based suggestions. Deeper sentiment analysis may be made possible through the substitution of the existing binary sentiment attribute with a new Sentiment-Analysis table, where the probabilistic or multi-class result(s) can be stored without altering the Reviews schema. Such extensions can be added with minimum disturbance because the database is in a modular format.

Finally, the database achieves its original goal of aggregating critic-based reviews as well as being consistent and allowing complex queries. Although execution might need optimisation based on indexing or refinement of queries at scale, our present design focuses on correctness and extensibility. The POPCORN_INDEX database offers a structurally sound and extensible base of a review aggregation system. The design supports the present functionality and realistic future additions by integrating high levels of normalisation, implied integrity and inbuilt analytical functionality.

Acknowledgements

I confirm that this project and the report are my own work. The entire database design, implementation of the SQL and the discussion that were written were all invented by me. Any outside materials used have been properly and clearly cited.

I know that plagiarism such as the use of the work or ideas of another person without giving the person due credit is a grave academic crime. I have ensured that none of the information contained in this report is plagiarism or that it is duly quoted.

The generation of a part of the data, namely the reviews, review_votes, and scores table values, was done with the help of artificial intelligence to fill the database with realistic sample data. The data was only utilized in testing and demonstration, and all database design, implementation and

analysis were done independently. The ER diagram included in this report was manually created by me using the Draw.io designing platform and was not generated by AI tools.

I would also like to note the support that was given by the lectures and course materials in COMP40725 that helped me to grasp the concepts of SQL programming. I also acknowledge that my background knowledge of SQL and relational database concepts was learned in my undergraduate years, thus being able to take up and complete such a sizable project.

There are no other people involved directly in the creation of this work.

References

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

Stonebraker, M., Abadi, D. J., DeWitt, D. J., et al. (2018). The end of an architectural era (it's time for a complete rewrite). *VLDB*.

Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases*.

Chen, P. P. (1976). The entity-relationship model—toward a unified view of data. *ACM TODS*.

Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM TODS*.

Bernstein, P. A. (1976). Synthesizing third normal form relations from functional dependencies. *ACM TODS*.

Kent, W. (1983). A simple guide to five normal forms in relational database theory. *Communications of the ACM*.

Widom, J. (1995). Research problems in data warehousing. *CIKM*.